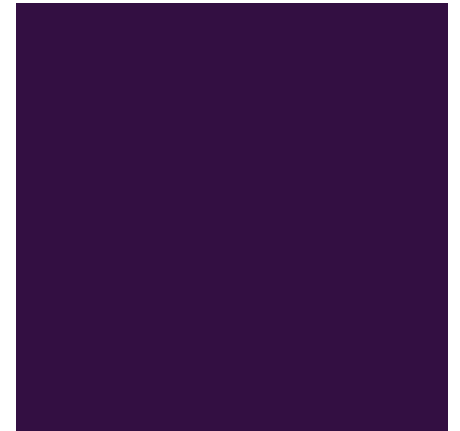
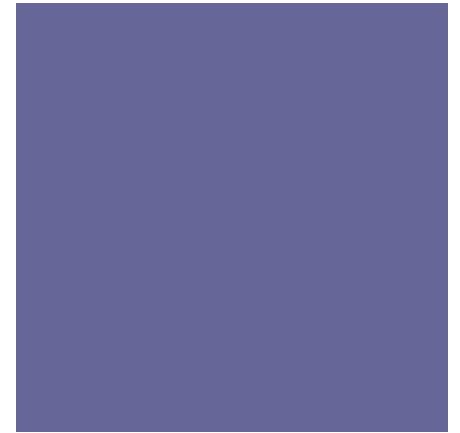




Kiến trúc máy tính
Bộ môn Kỹ thuật máy tính và mạng



+ Chương 10

Tập lệnh:

Đặc điểm và chức năng

+ Chương 10. Tập lệnh

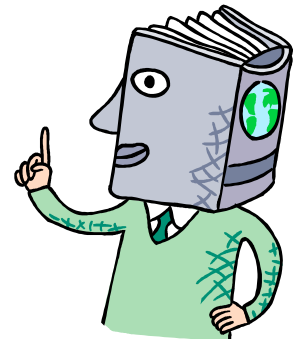


1. Các đặc điểm của lệnh máy
 - a. Thành phần của lệnh máy
 - b. Biểu diễn lệnh
 - c. Các loại lệnh
 - d. Số lượng địa chỉ
 - e. Thiết kế tập lệnh
2. Các kiểu toán hạng
 - a. Số
 - b. Ký tự
 - c. Dữ liệu logic
3. Các kiểu dữ liệu Intel x86 và ARM
 - a. Các kiểu dữ liệu x86
 - b. Các kiểu dữ liệu ARM
4. Các loại hoạt động
 - a. Truyền dữ liệu
 - b. Số học
 - c. Logic
 - d. Chuyển đổi
 - e. Vào/ra
 - f. Điều khiển hệ thống
 - g. Truyền điều khiển
5. Các loại hoạt động Intel x86 và ARM
 - a. Các loại hoạt động x86
 - b. Các loại hoạt động ARM



10.1 Đặc điểm tập lệnh

- Hoạt động (operation) của VXL được quyết định bởi các lệnh nó thực hiện, đó là các lệnh máy tính (*machine instructions* hay *computer instructions*)
 - VD: Lệnh STORE: lưu trữ dữ liệu vào bộ nhớ
- Tập hợp các lệnh khác nhau mà VXL có thể thực hiện được gọi là **tập lệnh (*instruction set*)** của VXL
- Mỗi lệnh phải có những thông tin cần thiết cho bộ xử lý thực hiện hoạt động
 - VD: Lệnh STORE ở trên phải đi kèm địa chỉ ngăn nhớ mà dữ liệu được ghi vào



Biểu đồ chu kỳ lệnh

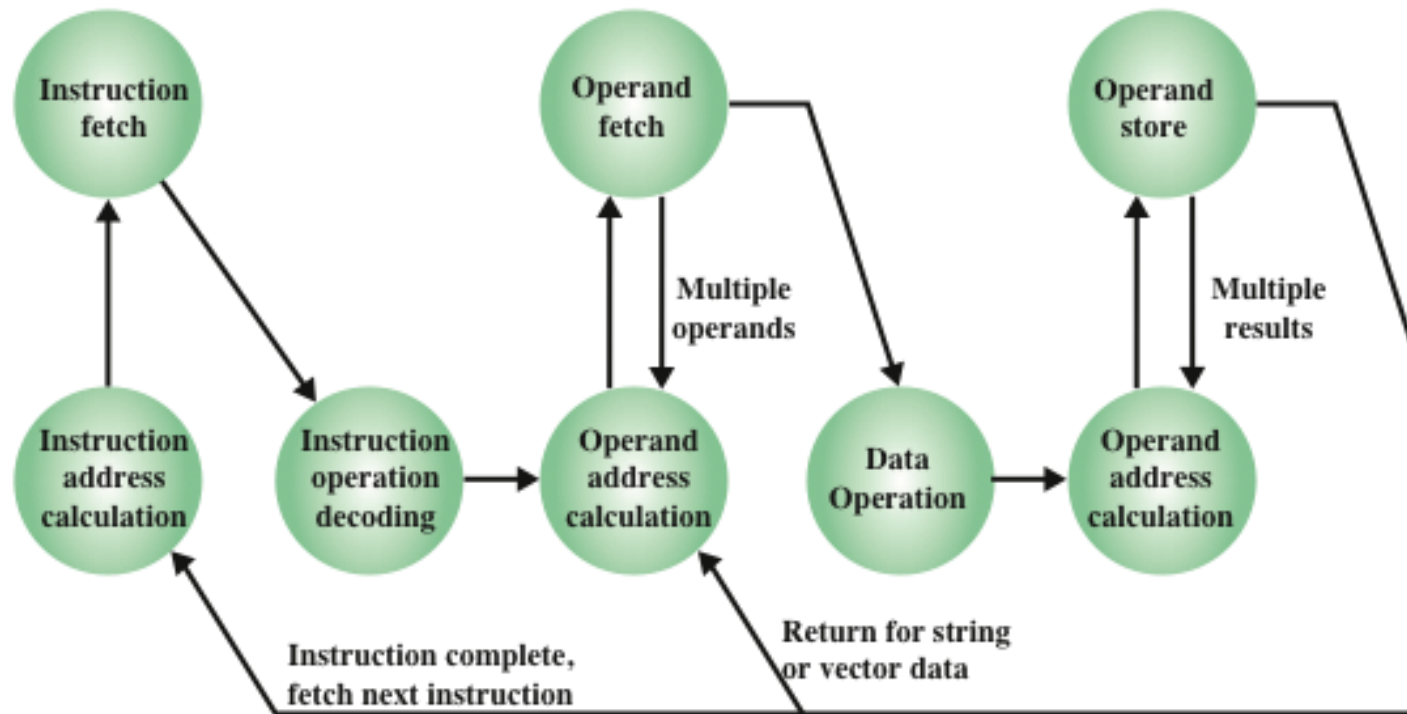


Figure 12.1 Instruction Cycle State Diagram

+ a. Các thành phần của lệnh

- **Mã lệnh** - Operation code (opcode): chỉ ra hoạt động (operation: hoạt động/phép toán) được thực hiện thông qua một mã nhị phân được gọi là mã lệnh (opcode)
- *Tham chiếu toán hạng nguồn*: Mỗi hoạt động có thể tham chiếu đến một hoặc nhiều toán hạng để lấy dữ liệu đầu vào cho hoạt động: các toán hạng này được gọi là **toán hạng nguồn**
- *Tham chiếu toán hạng kết quả (toán hạng đích)*: Hoạt động có thể đưa ra một kết quả
- *Tham chiếu lệnh tiếp theo*: Một số hoạt động có thể rẽ nhánh đến một câu lệnh ở vị trí khác → nói cho VXL nơi để lấy lệnh tiếp theo sau khi việc thực thi lệnh hiện tại hoàn thành
- VD: lệnh của máy IAS (20b: 8b opcode và 12b địa chỉ toán hạng)

Mã lệnh (Opcode)	Mô tả hợp ngữ	Công việc
00100001	STOR M(X)	Truyền dữ liệu từ thanh ghi AC vào ngăn nhớ có địa chỉ X trong bộ nhớ (M)
VD: 00100001 0001 1010 0000	STOR M(1A0)	Truyền dữ liệu từ AC vào ngăn nhớ có địa chỉ 1A0 trong bộ nhớ. AC: toán hạng nguồn (ngầm định) 1A0: toán hạng đích



Các toán hạng nguồn và kết quả có thể ở một trong bốn vùng sau:

1) Bộ nhớ chính hoặc bộ nhớ ảo

- Toán hạng nguồn/kết quả có thể là một vị trí bộ nhớ chính hoặc bộ nhớ ảo. Trong lệnh phải chứa địa chỉ bộ nhớ chính hoặc bộ nhớ ảo của toán hạng đó

2) Thanh ghi

- Toán hạng nguồn hoặc kết quả có thể là các thanh ghi. Một VXL chứa một hoặc nhiều thanh ghi, mỗi thanh ghi được gán cho một tên hoặc số riêng.
- Một lệnh có thể tham chiếu đến các thanh ghi này.

3) Tức thì

- Giá trị của toán hạng có thể được đưa trực tiếp vào trong câu lệnh

4) Thiết bị vào/ra (I/O)

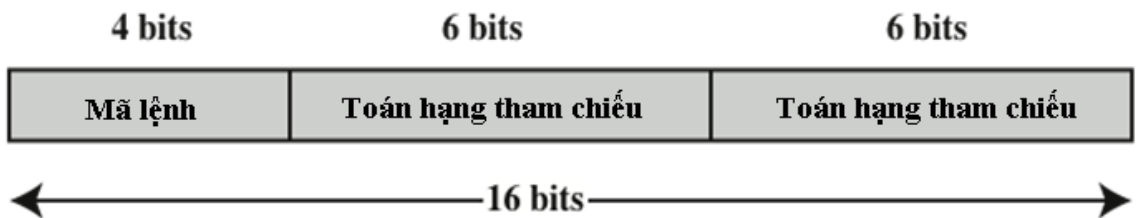
- Dữ liệu có thể lấy từ (hoặc ghi ra) một thiết bị I/O → lệnh phải chỉ ra địa chỉ thiết bị và module vào/ra tương ứng



b. Biểu diễn lệnh

- Trong máy tính, mỗi câu lệnh được biểu diễn bằng một chuỗi bit nhị phân
- Câu lệnh được chia ra thành các trường tương ứng với các thành phần cấu thành của lệnh

- Ví dụ



- Trong các tài liệu, để dễ hiểu, lệnh thường được biểu diễn dưới dạng các ký hiệu thay vì các bit nhị phân. Opcode được viết tắt, mô tả hoạt động (phép toán).

Ví dụ: Tập lệnh IAS

ADD	Cộng
SUB	Trừ
MUL	Nhân
DIV	Chia
LOAD	Tải dữ liệu từ bộ nhớ
STOR	Lưu dữ liệu vào bộ nhớ



Ví dụ:

NNLT bậc cao được đưa ra để giúp công việc của lập trình viên thuận lợi hơn

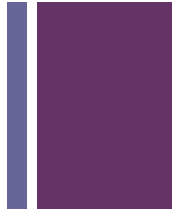
Ví dụ câu lệnh: $X = X + Y$ viết bằng NN C++ nếu dịch sang tập lệnh IAS sẽ gồm các lệnh như sau

0010 0000 0001 0000	LOAD X	Đọc X từ bộ nhớ vào thanh ghi AC
0101 0000 0001 0001	ADD Y	Đọc Y từ bộ nhớ, cộng Y với AC, kết quả ghi vào AC
1000 0000 0001 0000	STOR X	Ghi AC vào X trong bộ nhớ

Trong đó: X, Y là biến, có địa chỉ BN: X: 0000 0001 0000
Y: 0000 0001 0001



c. Các loại lệnh: chia thành 4 nhóm

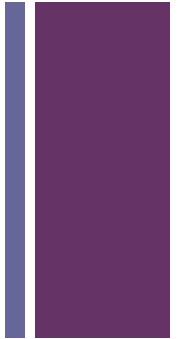


- Xử lý dữ liệu: các lệnh số học và logic
 - Các lệnh số học cung cấp khả năng tính toán để xử lý dữ liệu số
 - Các lệnh logic (Boolean) hoạt động trên các bit, cung cấp khả năng xử lý bất kỳ loại dữ liệu nào. Các lệnh này chủ yếu thực thi với các bit trên thanh ghi
- Lưu trữ dữ liệu
 - Các lệnh đọc/ghi dữ liệu từ/vào thanh ghi hoặc bộ nhớ
- Di chuyển dữ liệu:
 - Gồm các lệnh vào/ra: được sử dụng để truyền chương trình và dữ liệu vào bộ nhớ và các kết quả tính toán được trả lại cho người dùng
Ví dụ: chương trình user và dữ liệu (lưu trữ ở ổ cứng) được nạp vào RAM
- Điều khiển: gồm các lệnh kiểm tra và rẽ nhánh
 - Các lệnh kiểm tra được sử dụng để kiểm tra giá trị của dữ liệu hoặc trạng thái của một phép toán
 - Các lệnh rẽ nhánh được dùng để rẽ nhánh tập lệnh khác nhau tùy thuộc vào **điều kiện** cụ thể



d. Số lượng các địa chỉ

- Một thuộc tính quan trọng của tập lệnh là **số lượng địa chỉ**
- Tùy thuộc vào các lệnh khác nhau sẽ có số lượng toán hạng khác nhau
 - Như phần trên đã đề cập, các toán hạng có thể là các vị trí nhớ (trong bộ nhớ chính) hoặc I/O (I/O port), được đặc trưng bởi một địa chỉ logic
- Vậy, số lượng địa chỉ tối đa trong một lệnh là bao nhiêu:
 - Các lệnh số học và logic: cần tối đa 4 địa chỉ: 2 đ/c toán hạng nguồn, 1 đ/c toán hạng đích, 1 đ/c toán hạng truy xuất câu lệnh tiếp theo.
 - Số lượng địa chỉ càng nhiều thì kích thước lệnh càng lớn
 - Với hầu hết các hệ VXL, số lượng địa chỉ là 1, 2 hoặc 3. Lệnh tiếp theo được ngầm định truy xuất thông qua thanh ghi PC (program counter register)



+ Số lượng các địa chỉ (tiếp)



Ví dụ các lệnh tính toán biểu thức $Y=(A-B)/(C+(D\times E))$ trong 3 trường hợp:

■ Trường hợp lệnh 3 địa chỉ:

- Kích thước lệnh dài
- Toán hạng 1, toán hạng 2, kết quả
- T: vị trí bộ nhớ tạm thời để lưu trữ kết quả

Instruction		Comment
SUB	Y, A, B	$Y \leftarrow A - B$
MPY	T, D, E	$T \leftarrow D \times E$
ADD	T, T, C	$T \leftarrow T + C$
DIV	Y, Y, T	$Y \leftarrow Y \div T$

(a) Three-address instructions

■ Trường hợp lệnh 2 địa chỉ:

- Kết quả phép toán được ghi vào một địa chỉ

Instruction		Comment
MOVE	Y, A	$Y \leftarrow A$
SUB	Y, B	$Y \leftarrow Y - B$
MOVE	T, D	$T \leftarrow D$
MPY	T, E	$T \leftarrow T \times E$
ADD	T, C	$T \leftarrow T + C$
DIV	Y, T	$Y \leftarrow Y \div T$

(b) Two-address instructions

Ví dụ các lệnh tính toán biểu thức $Y = (A - B)/(C + (D \times E))$ trong 3 trường hợp

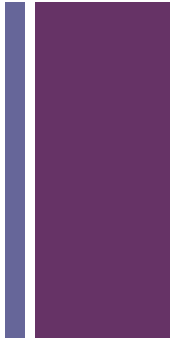
- Trường hợp lệnh 1 địa chỉ:
 - Một toán hạng ngầm định là thanh ghi AC
 - Phổ biến ở các hệ VXL đơn giản, đời đầu

Instruction	Comment
LOAD D	$AC \leftarrow D$
MPY E	$AC \leftarrow AC \times E$
ADD C	$AC \leftarrow AC + C$
STOR Y	$Y \leftarrow AC$
LOAD A	$AC \leftarrow A$
SUB B	$AC \leftarrow AC - B$
DIV Y	$AC \leftarrow AC \div Y$
STOR Y	$Y \leftarrow AC$

(c) One-address instructions

- Trường hợp lệnh 0 địa chỉ:
 - Ngầm định 2 ngăn nhớ ở đỉnh vùng ngăn xếp (stack) của BN

- Số lượng các địa chỉ càng ít thì số lượng các câu lệnh để tính toán biểu thức càng nhiều



Bảng 10.1

Mô tả các lệnh 0, 1, 2, 3 địa chỉ

Number of Addresses	Symbolic Representation	Interpretation
3	OP A, B, C	$A \leftarrow B \text{ OP } C$
2	OP A, B	$A \leftarrow A \text{ OP } B$
1	OP A	$AC \leftarrow AC \text{ OP } A$
0	OP	$T \leftarrow (T - 1) \text{ OP } T$

AC = accumulator

T = top of stack

(T - 1) = second element of stack

A, B, C = memory or register locations

- 
- + ■ Số lượng địa chỉ trong mỗi lệnh là một yếu tố cơ bản đối với thiết kế VXL.
 - Lệnh càng ít địa chỉ → kích thước lệnh ngắn hơn → VXL ít phức tạp hơn → chương trình cần nhiều lệnh hơn để thực hiện một công việc → mất thời gian hơn
 - Lệnh một địa chỉ, lập trình viên thường chỉ có sẵn một thanh ghi đa năng: *thanh ghi AC*.
 - Với các hệ VXL cho phép lệnh nhiều địa chỉ thường có nhiều thanh ghi đa năng. Điều này cho phép một số hoạt động được thực hiện chỉ trong các thanh ghi → không cần truy xuất BNC → tốc độ nhanh hơn.
- Hầu hết các hệ VXL hiện đại sử dụng kết hợp các cấu trúc lệnh hai và ba địa chỉ.

e. Thiết kế tập lệnh



- Tập lệnh định nghĩa các chức năng được thực hiện bởi VXL
- Là phương tiện của người lập trình trong việc điều khiển VXL
- Các vấn đề thiết kế cơ bản:
 - **Danh sách các hoạt động:** bao nhiêu hoạt động và hoạt động nào được đưa ra? Độ phức tạp của các hoạt động như thế nào?
 - **Các kiểu dữ liệu:** các kiểu dữ liệu mà các hoạt động tham chiếu đến
 - **Cấu trúc lệnh:** độ dài lệnh theo bit, số lượng địa chỉ, kích thước của các trường khác nhau, v.v ...
 - **Các thanh ghi:** số lượng các thanh ghi của VXL có thể được tham chiếu đến bởi lệnh và chức năng của chúng
 - **Chế độ địa chỉ:** các cách để định ra địa chỉ của toán hạng

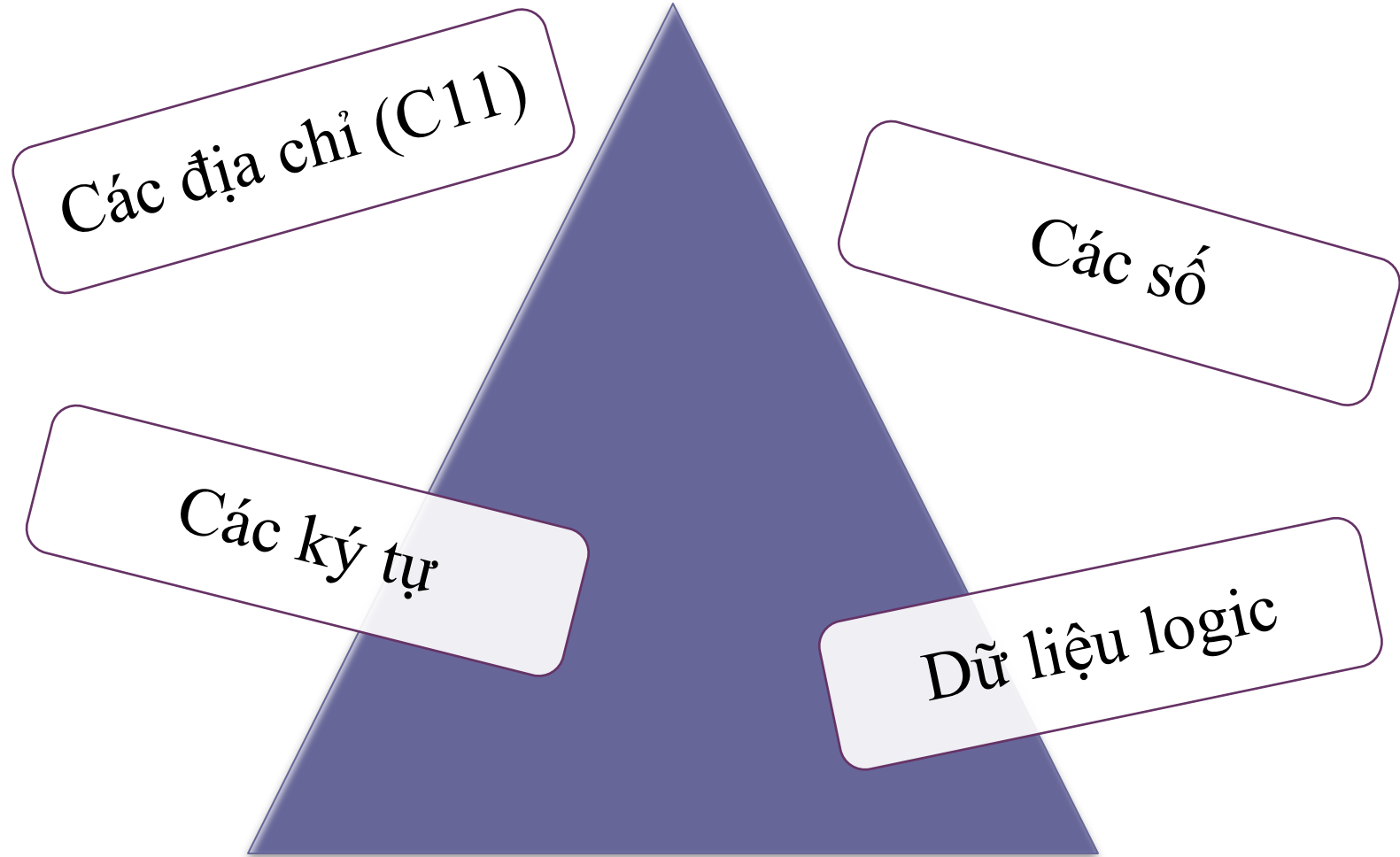
+ Ví dụ

- Viết chương trình tính giá trị biểu thức sau sử dụng các tập lệnh 0, 1, 2, 3 địa chỉ cho ở bảng dưới

$$X = (A + B \times C) / (D - E \times F)$$

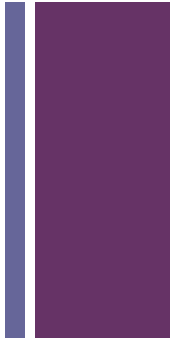
0 Address	1 Address	2 Address	3 Address
PUSH M	LOAD M	MOVE ($X \leftarrow Y$)	MOVE ($X \leftarrow Y$)
POP M	STORE M	ADD ($X \leftarrow X + Y$)	ADD ($X \leftarrow Y + Z$)
ADD	ADD M	SUB ($X \leftarrow X - Y$)	SUB ($X \leftarrow Y - Z$)
SUB	SUB M	MUL ($X \leftarrow X \times Y$)	MUL ($X \leftarrow Y \times Z$)
MUL	MUL M	DIV ($X \leftarrow X/Y$)	DIV ($X \leftarrow Y/Z$)
DIV	DIV M		

10.2 Các kiểu toán hạng





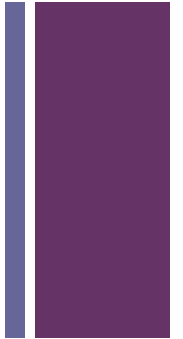
Dữ liệu kiểu số



- Tất cả các ngôn ngữ máy đều có dữ liệu dạng số
- Các số được lưu trữ trong máy tính đều hữu hạn:
 - Hữu hạn về độ lớn của các số biểu diễn trên máy
 - Hữu hạn về độ chính xác đối với số dấu phẩy động
- Ba kiểu dữ liệu số thông thường trong máy tính:
 1. Số nguyên nhị phân hoặc số nhị phân dấu chấm tĩnh
 2. Số nhị phân dấu chấm động
 3. Số thập phân đóng
 - Mỗi chữ số thập phân được biểu diễn bởi một mã 4 bit
$$0 = 0000, 1 = 0001, \dots, 8 = 1000 \text{ và } 9 = 1001$$
Dấu dương (+): 1100, dấu âm (-): 1101
- Chiều dài mã thường là bội của 8b



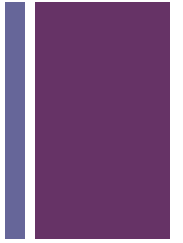
Dữ liệu kiểu ký tự



- Một trong những dạng dữ liệu cơ bản là văn bản (text) hoặc chuỗi ký tự (character strings)
- Dữ liệu văn bản dưới dạng ký tự không thể lưu trữ hoặc truyền qua hệ thống xử lý dữ liệu và truyền thông vì các hệ thống này được thiết kế cho dữ liệu nhị phân → sử dụng bảng mã
- Bảng mã mã hóa ký tự được sử dụng phổ biến nhất là bảng mã IRA (International Reference Alphabet)
 - Còn được gọi ở Mỹ là bảng mã ASCII (American Standard Code for Information Interchange)
- Một bảng mã khác được sử dụng để mã hóa các ký tự bảng mã EBCDIC (Extended Binary Coded Decimal Interchange Code) được sử dụng trong các máy mainframe của IBM



Dữ liệu logic



- Một khối n-bit gồm n phần tử dữ liệu 1 bit, mỗi item có giá trị 0 hoặc 1
- Hai ưu điểm của view theo hướng bit:
 - Đôi khi, ta muốn lưu trữ một mảng các bit nhị phân hoặc dữ liệu Boolean/nhị phân, trong đó mỗi phần tử chỉ nhận giá trị 1 (đúng) hoặc 0 (sai). Với kiểu dữ liệu logic, bộ nhớ lưu trữ điều này hiệu quả nhất
 - Trong một số trường hợp chúng ta cần thao tác với các bit
 - Trường hợp phép toán dấu chấm động: dịch các bit có nghĩa
 - Trường hợp chuyển đổi từ mã IRA thành mã thập phân đóng gói: trích xuất 4 bit bên phải của mỗi byte



10.3 Các kiểu dữ liệu Intel x86 và ARM

a. Intel x86

Kiểu dữ liệu	Mô tả
General	Các vị trí bộ nhớ kích thước byte, word (16 bits), doubleword (32 bits), quadword (64 bits), và double quadword (128 bits) với dữ liệu nhị phân bất kỳ
Integer	Giá trị nhị phân có dấu lưu trữ trong một byte, word, hoặc doubleword, sử dụng dạng biểu diễn bù 2
Ordinal	Một số nguyên không dấu lưu trữ trong một byte, word, hoặc doubleword.
Unpacked binary coded decimal (BCD)	Biểu diễn một ký tự BCD trong khoảng từ 0 đến 9, với mỗi ký tự dùng một byte
Packed BCD	Biểu diễn 2 ký tự BCD trong 1 byte, một packed BCD có dải giá trị từ 0 đến 99
Near pointer	Địa chỉ hiệu dụng 16-bit, 32-bit, hoặc 64-bit biểu diễn độ lệch (offset) trong một phân đoạn. Được sử dụng cho tất cả các con trỏ trong bộ nhớ không phân đoạn và cho các tham chiếu trong một đoạn của bộ nhớ phân đoạn
Far pointer	Địa chỉ logic gồm 16-bit trỏ tới một đoạn (segment) và một địa chỉ lệch offset 16, 32, hoặc 64 bits. Far pointers được sử dụng để tham chiếu bộ nhớ trong mô hình bộ nhớ phân đoạn
Bit field	Một chuỗi bit liên tục trong đó mỗi bit được coi như một đơn vị độc lập. Chuỗi bit có thể bắt đầu tại bất cứ vị trí nào trong bất cứ byte nào và có thể chứa tới 32 bit
Bit string	Một dãy bit liên tục, gồm từ 0 đến 232 - 1 bit.
Byte string	Một dãy byte, word hoặc doublewords liên tục gồm từ 0 đến 232 - 1 byte.
Floating point	Xem hình 10.4.
Packed SIMD (single instruction, multiple data)	Các kiểu dữ liệu Packed 64-bit and 128-bit



Định dạng dữ liệu số x86

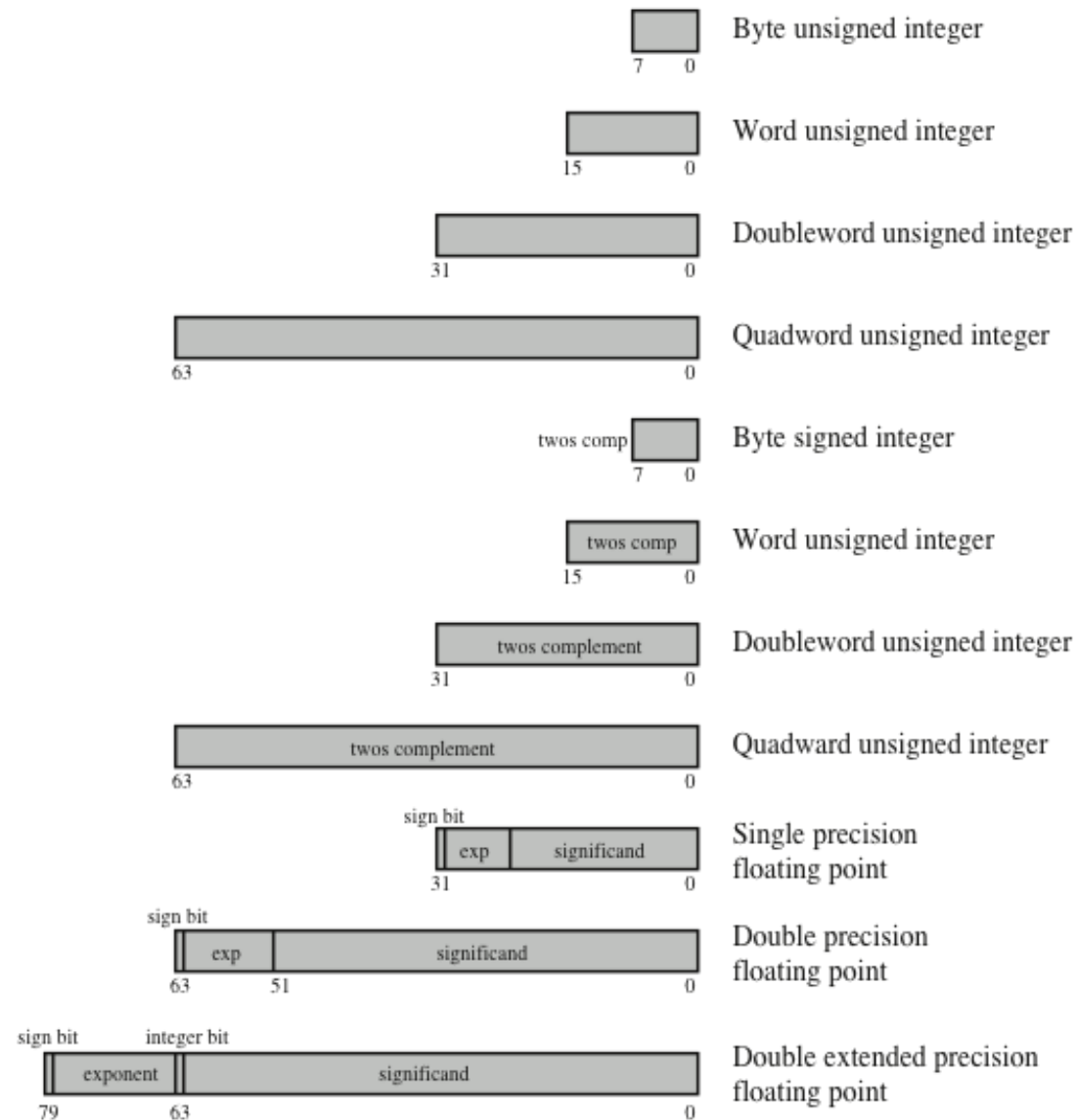
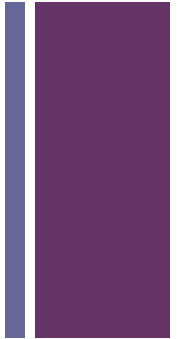


Figure 12.4 x86 Numeric Data Formats



Các kiểu dữ liệu SIMD (Single-Instruction-Multiple-Data)



- Dùng cho kiến trúc x86 như là 1 phần của mở rộng tập lệnh để tối ưu hóa hiệu năng của các ứng dụng đa phương tiện
- Các mở rộng tập lệnh gồm MMX (multimedia extensions) và SSE (streaming SIMD extensions)
- Các kiểu dữ liệu SIMD:
 - Byte đóng gói và số nguyên byte đóng gói
 - Word đóng gói và số nguyên word đóng gói
 - Doubleword đóng gói và số nguyên doubleword đóng gói
 - Quadword đóng gói và số nguyên quadword đóng gói
 - Packed single-precision floating-point and packed doubleprecision floating-point

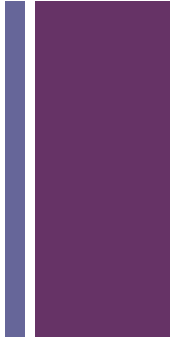


b. Các kiểu dữ liệu của ARM

- Vi xử lý ARM hỗ trợ các kiểu dữ liệu có kích thước:
 - 8b (byte)
 - 16b (halfword)
 - 32b (word)
- Đối với tất cả ba kiểu dữ liệu, có một kiểu tương ứng dành cho số nguyên không dấu (số nguyên dương)
- Tất cả ba kiểu dữ liệu cũng có thể được sử dụng cho số nguyên biểu diễn bù 2
- VXL ARM không hỗ trợ phần cứng cho biểu diễn dấu chấm động. Các phép toán cho dấu chấm động phải được thực hiện bằng phần mềm



Hỗ trợ chuyển đổi Endian trong ARM



- Khái niệm Endian: cách tổ chức dữ liệu trong bộ nhớ máy tính
- Có 2 loại endian:
 - Little Endian: byte có giá trị nhỏ nhất lưu trữ ở vị trí nhớ có địa chỉ nhỏ nhất
 - Big Endian: byte có giá trị lớn nhất lưu trữ ở vị trí nhớ có địa chỉ nhỏ nhất
- ARM cho phép chuyển đổi giữa hai dạng endian: sử dụng E-bit trong thanh ghi PS(thanh ghi trạng thái chương trình)
 - E-bit = 1: Big endian
 - E-bit = 0: Little endian

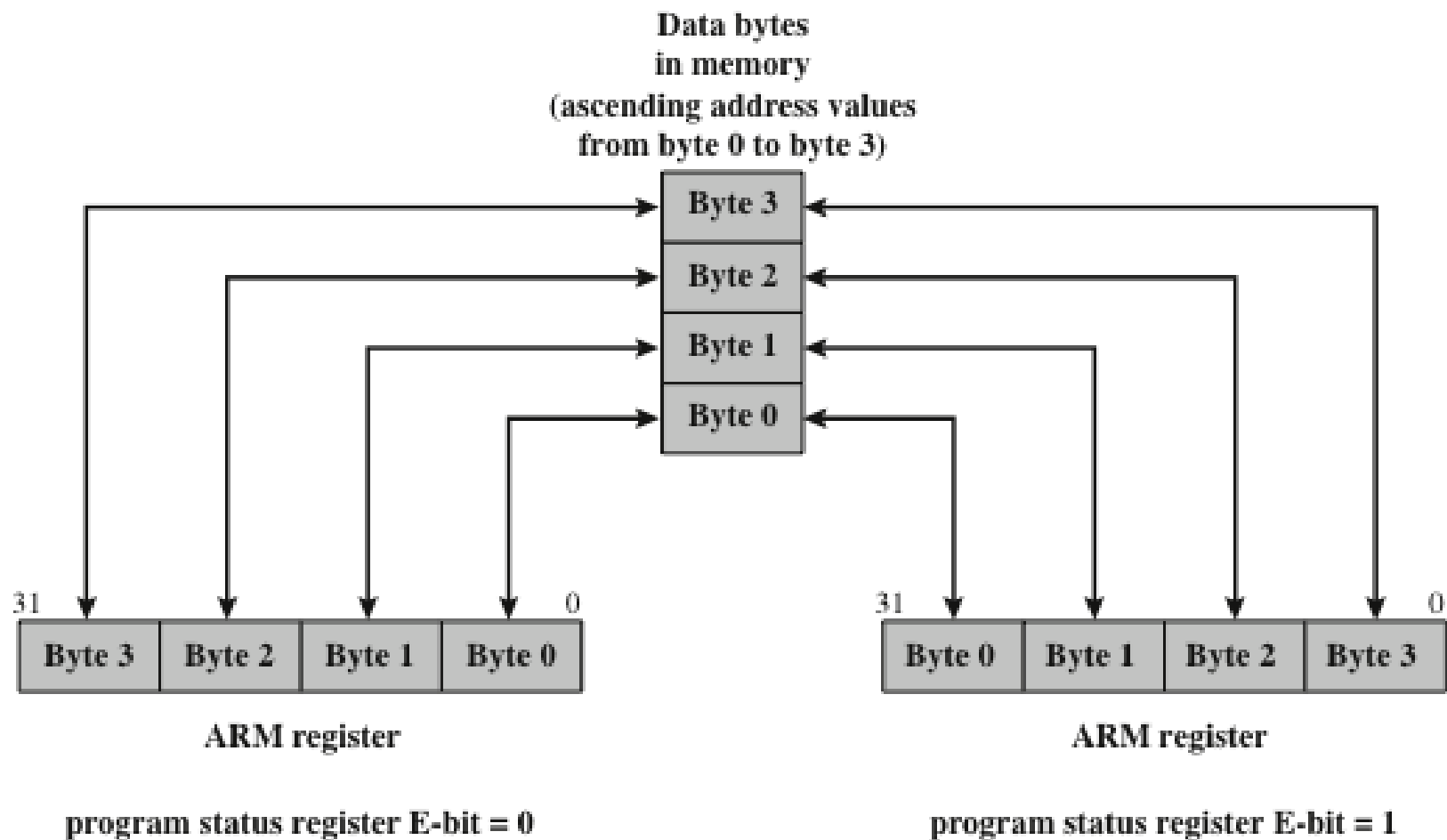
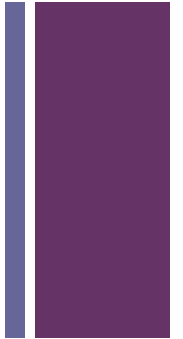


Figure 12.5 ARM Endian Support - Word Load/Store with E-bit



10.4 Các loại operation (hoạt động, phép toán)



- Có rất nhiều các loại lệnh khác nhau đối với mỗi thể hệ máy tính. Tuy nhiên, một số loại chung đối với tất cả các máy tính như sau:
 - a. Các lệnh truyền dữ liệu
 - b. Các lệnh tính toán số học
 - c. Các lệnh logic
 - d. Các lệnh chuyển đổi
 - e. Các lệnh vào/ra
 - f. Các lệnh điều khiển hệ thống
 - g. Các lệnh truyền điều khiển

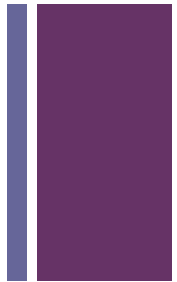
a. Truyền dữ liệu

- Đây là lệnh cơ bản nhất của tất cả các hệ máy tính
- Cần phải định rõ:
 - Địa chỉ của các toán hạng nguồn và đích, loại vị trí: bộ nhớ, thanh ghi, ngăn xếp
 - Kích thước của dữ liệu được truyền
 - Chế độ định địa chỉ đối với mỗi toán hạng
- Các lệnh cơ bản

Truyền dữ liệu	MOVE	Copy dữ liệu từ nguồn đến đích
	LOAD	Nạp dữ liệu từ bộ nhớ đến bộ xử lý
	STORE	Cất dữ liệu từ bộ xử lý đến bộ nhớ
	EXCHANGE	Trao đổi nội dung của nguồn và đích
	CLEAR	Chuyển các bit 0 vào toán hạng đích
	SET	Chuyển các bit 1 vào toán hạng đích
	PUSH	Cất nội dung toán hạng nguồn vào ngăn xếp
	POP	Lấy nội dung đỉnh ngăn xếp đưa đến toán hạng đích



b. Số học



- Hầu hết các máy đều cung cấp các phép toán số học cơ bản: cộng, trừ, nhân, chia
- Cố định với toán hạng là số nguyên có dấu. Một số máy có các phép toán trên với:
 - Số thực dấu chấm động
 - Số thập phân đóng
- Một số phép toán khác có dạng lệnh một toán hạng:
 - Tuyệt đối: Tính giá trị tuyệt đối của một toán hạng
 - Phép đảo: Đổi dấu một toán hạng
 - Phép tăng: Cộng toán hạng thêm 1 đơn vị
 - Phép giảm: Trừ toán hạng đi 1 đơn vị



Một số lệnh số học

Xử lý số học	ADD	Cộng hai toán hạng
	SUBTRACT	Trừ hai toán hạng
	MULTIPLY	Nhân hai toán hạng
	DIVIDE	Chia hai toán hạng
	ABSOLUTE	Lấy trị tuyệt đối toán hạng
	NEGATE	Đổi dấu toán hạng (lấy bù 2)
	INCREMENT	Tăng toán hạng thêm 1
	DECREMENT	Giảm toán hạng đi 1

c. Phép toán logic (luận lý)

■ Các phép toán logic cơ bản

Xử lý logic	AND	Thực hiện phép AND hai toán hạng
	OR	Thực hiện phép OR hai toán hạng
	NOT	Đảo bit của toán hạng (lấy bù 1)
	XOR	Thực hiện phép XOR hai toán hạng
	TEST	Kiểm tra điều kiện cụ thể; thiết lập cờ dựa trên kết quả
	COMPARE	So sánh logic hoặc số học của hai hoặc nhiều toán hạng; thiết lập cờ dựa trên kết quả
	SHIFT	Dịch trái (phải) toán hạng
	ROTATE	Quay vòng trái (phải) toán hạng

P	Q	NOT P	P AND Q	P OR Q	P XOR Q	P=Q
0	0	1	0	0	0	1
0	1	1	0	1	1	0
1	0	0	0	1	1	0
1	1	0	1	1	0	1

+ ■ Cho phép thực hiện với các khối n-bit

(R1) = 10100101

(R2) = 00001111

(R1) = 10100101

(R2) = 11111111

(R1) AND (R2) = 00000101

(R1) XOR (R2) = 01011010

■ Ngoài ra, gồm có một số phép toán dịch và xoay vòng

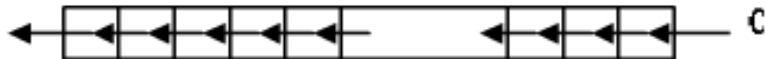
SHIFT

Dịch trái (phải) toán hạng

ROTATE

Quay trái (phải) toán hạng

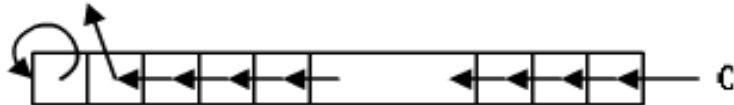
Dịch trái logic



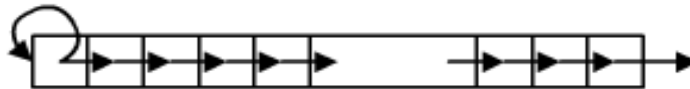
Dịch phải logic



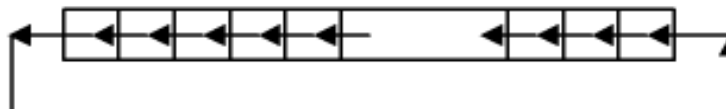
Dịch trái số học



Dịch phải số học



Quay trái logic





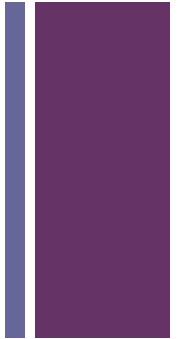
Input	Operation	Result
10100110	Logical right shift (3 bits)	00010100
10100110	Logical left shift (3 bits)	00110000
10100110	Arithmetic right shift (3 bits)	11110100
10100110	Arithmetic left shift (3 bits)	10110000
10100110	Right rotate (3 bits)	11010100
10100110	Left rotate (3 bits)	00110101

Ví dụ về các phép toán dịch và xoay vòng



d. Chuyển đổi

- Lệnh chuyển đổi: thay đổi định dạng hoặc tác động vào định dạng của dữ liệu
 - Ví dụ 1: chuyển đổi từ nhị phân sang mã thập phân đóng
 - Ví dụ 2: chuyển đổi từ mã IRA sang mã EBCDIC qua một bảng gồm 256 byte trong bộ nhớ chính.



+e. Vào/ra

■ Các lệnh vào/ra liên quan đến:

■ Cơ chế địa chỉ:

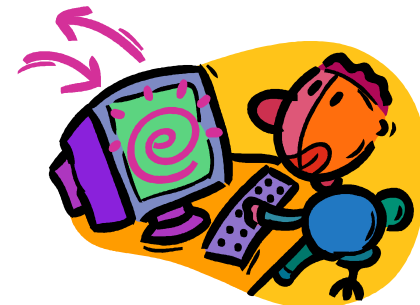
- I/O chương trình, ánh xạ riêng biệt - Isolated programmed I/O

- I/O chương trình, ánh xạ bộ nhớ - Memory-mapped programmed I/O

■ Cơ chế DMA

■ Cơ chế điều khiển I/O sử dụng bộ xử lý vào ra

■ Nhiều tập lệnh của các hệ VXL chỉ cung cấp một vài lệnh I/O với các hoạt động cụ thể được xác định bởi các tham số, các mã hoặc các từ lệnh





Điều khiển vào/ra	INPUT	Truyền dữ liệu từ một cổng hoặc thiết bị I/O xác định đến đích (VD: Bộ nhớ chính hoặc thanh ghi bộ xử lý)
	OUTPUT	Truyền dữ liệu từ nguồn xác định đến cổng hoặc thiết bị I/O
	START I/O	Truyền lệnh đến bộ xử lý I/O để bắt đầu hoạt động I/O
	TEST I/O	Truyền thông tin trạng thái từ hệ thống I/O đến đích xác định

f. Điều khiển hệ thống

- Các câu lệnh có thể được thực hiện chỉ khi VXL trong trạng thái đặc quyền hoặc đang thực hiện một chương trình trong vùng đặc quyền đặc biệt của bộ nhớ.
- Thông thường các lệnh này được dành riêng cho hệ điều hành
- Ví dụ về các hoạt động điều khiển hệ thống:
 - Một câu lệnh điều khiển hệ thống có thể đọc hoặc thay đổi thanh ghi điều khiển
 - Câu lệnh để đọc hoặc thay đổi khóa bảo vệ bộ nhớ
 - Truy cập vào các khối điều khiển tiến trình trong hệ thống đa chương trình

+ g. Truyền điều khiển

- Các hoạt động truyền điều khiển là cần thiết:
 - Cần thiết để có thể thực thi mỗi câu lệnh nhiều hơn một lần
 - Hầu như mọi chương trình đều gồm có việc ra quyết định
 - Cơ chế phân tách các nhiệm vụ ra thành các công việc nhỏ hơn có thể thực hiện tại các thời điểm khác nhau
- Các hoạt động truyền điều khiển nói chung:
 - Rẽ nhánh:
 - Rẽ nhánh có điều kiện: chỉ rẽ nhánh (thiết lập địa chỉ (toán hạng của lệnh) vào thanh ghi PC) khi một điều kiện nhất định được thỏa mãn
 - Rẽ nhánh không điều kiện: việc rẽ nhánh luôn được thực hiện.
 - Lệnh nhảy
 - Lệnh gọi thủ tục

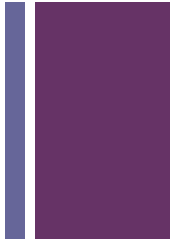
+ Truyền điều khiển

■ Các lệnh nhảy và gọi thủ tục

Truyền điều khiển	JUMP (BRANCH)	Truyền không điều kiện; tải địa chỉ xác định vào PC
	JUMP CONDITIONAL	Kiểm tra điều kiện cụ thể; tải địa chỉ xác định vào PC hoặc không làm gì tùy thuộc vào điều kiện
	CALL	Cất nội dung của PC (địa chỉ trở về) vào một vị trí xác định; Nạp địa chỉ lệnh đầu tiên của chương trình con vào PC
	RETURN	Đặt địa chỉ trở về trả lại cho PC để trở về chương trình chính
	SKIP	Tăng PC để bỏ qua lệnh tiếp theo
	SKIP CONDITIONAL	Kiểm tra điều kiện cụ thể; bỏ qua lệnh hoặc không làm gì tùy thuộc vào điều kiện
	HALT	Dừng thực thi chương trình
	WAIT (HOLD)	Dừng thực thi chương trình; liên tục kiểm tra điều kiện cụ thể; quay lại thực thi tiếp khi điều kiện được thỏa mãn
	NO OPERATION	Không có hành động nào được thực hiện, nhưng việc thực thi chương trình vẫn được tiếp tục



Ví dụ một số lệnh rẽ nhánh

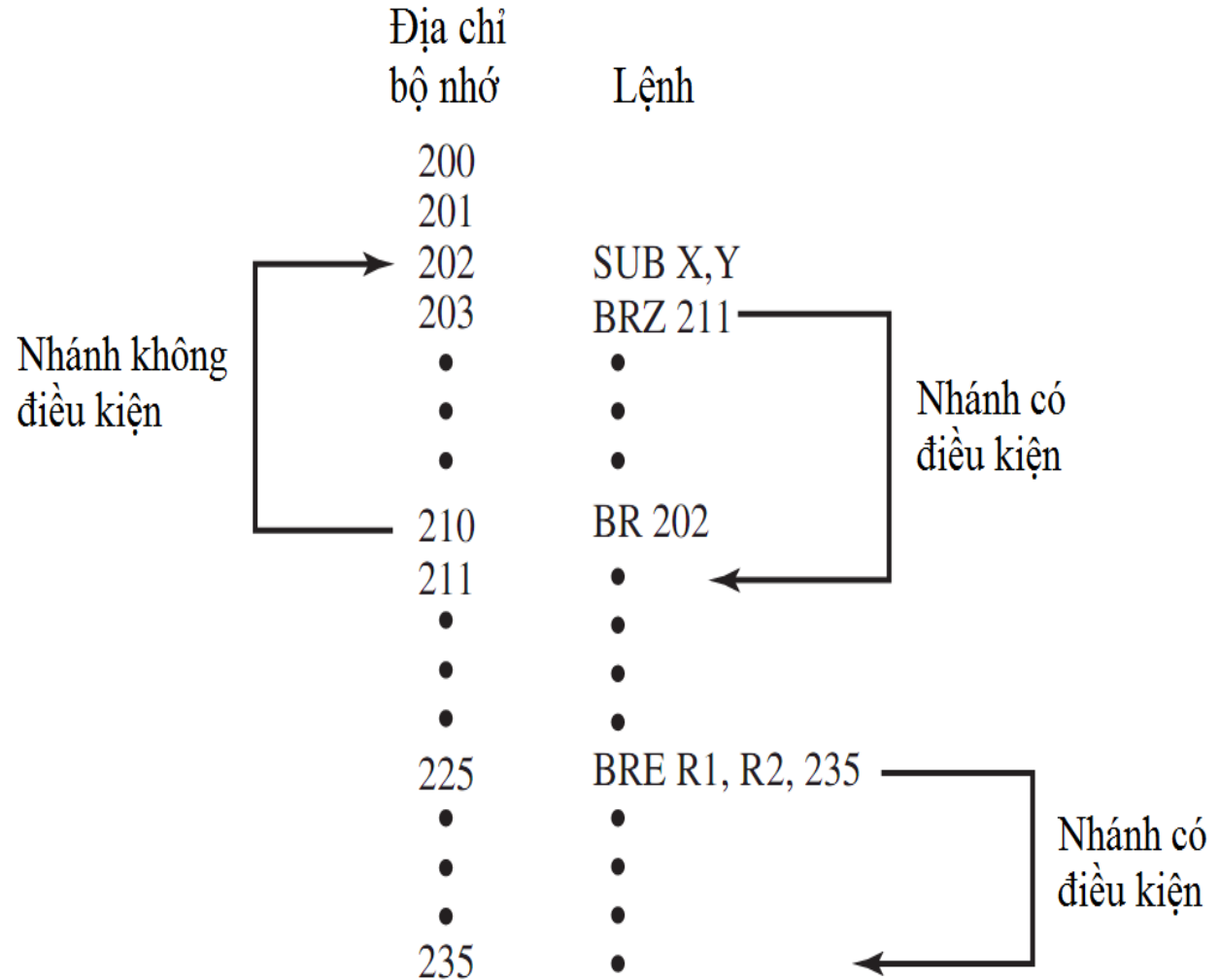


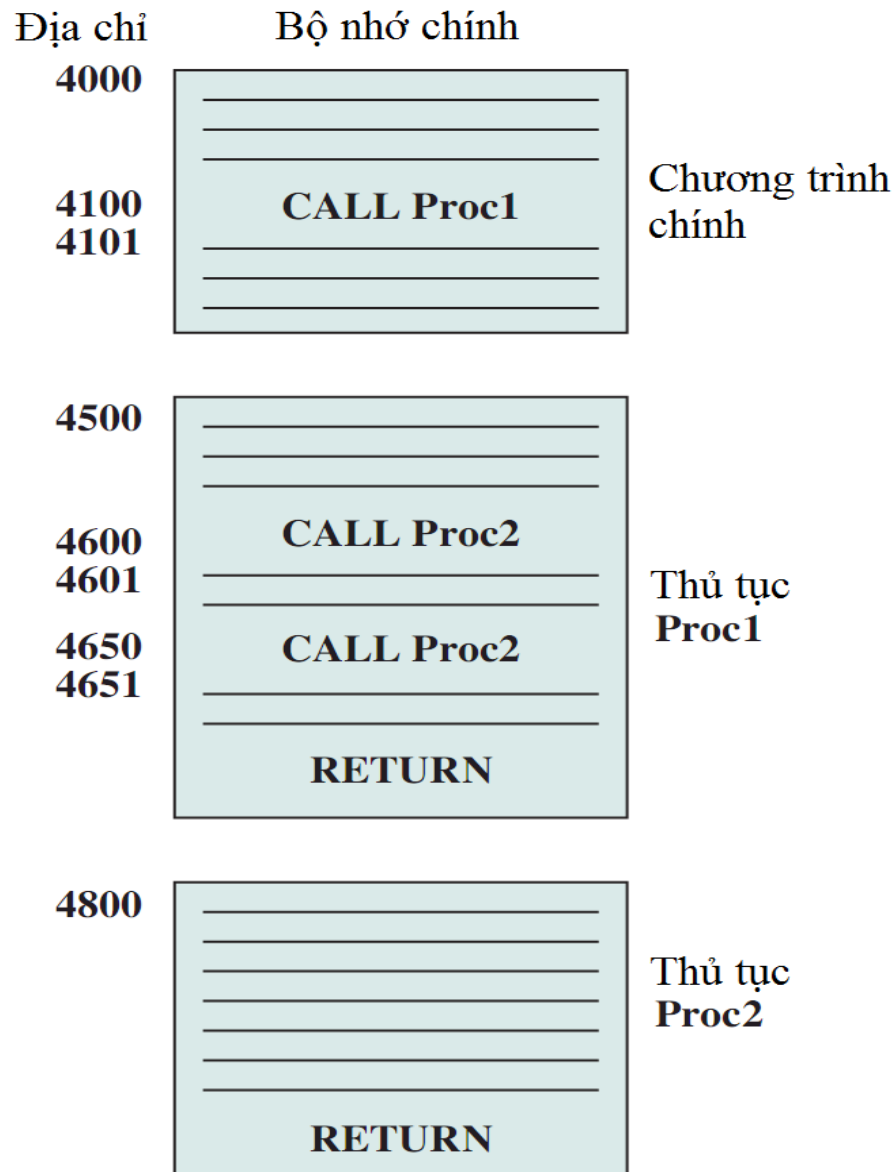
- BRP X: Rẽ nhánh đến vị trí X nếu kết quả dương.

- BRN X: Rẽ nhánh đến vị trí X nếu kết quả âm.

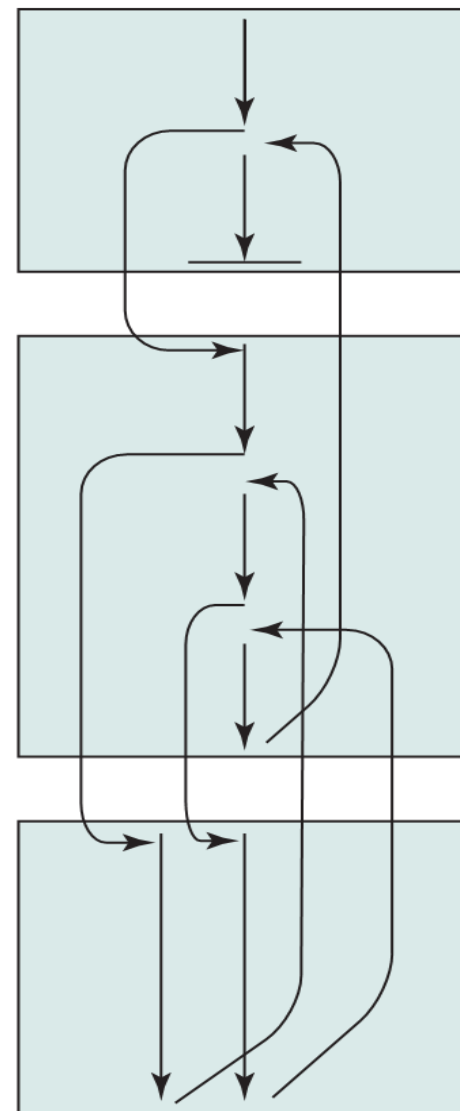
- BRZ X: Rẽ nhánh đến vị trí X nếu kết quả là 0.

- BRO X: Rẽ nhánh đến vị trí X nếu xảy ra tràn.





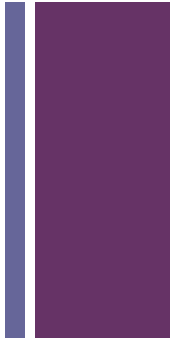
(a) Lệnh Call và Return



(b) Chuỗi thực thi



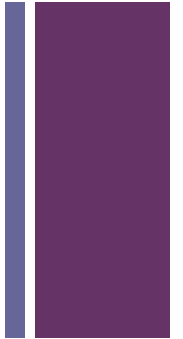
Hành động của VXL đối với các loại hoạt động



- Truyền dữ liệu:
 - Truyền dữ liệu từ một vị trí đến vị trí khác
 - Nếu gồm bộ nhớ:
 - Xác định địa chỉ bộ nhớ
 - Thực hiện việc chuyển địa chỉ ảo sang đ/c thực tế
 - Kiểm tra cache
 - Bắt đầu hoạt động đọc/ghi
- Tính toán số học
 - Có thể bao gồm cả hoạt động truyền dữ liệu (trước hoặc sau khi tính toán)
 - Các phép toán được thực hiện trong ALU
 - Thiết lập các mã điều kiện và các cờ



Hành động của VXL đối với các loại hoạt động (tiếp)



- Tính toán logic: giống tính toán số học
- Chuyển đổi: tương tự như tính toán số học và logic. Có thể gồm một logic đặc biệt để thực hiện chuyển đổi
- Truyền điều khiển:
 - Cập nhật thanh ghi PC. Với lời gọi/trả về chương trình con, quản lý các thông số đi qua hoặc liên kết
- Vào/ra
 - Đưa ra các lệnh cho module I/O
 - Trong chế độ memory-mapped I/O, xác định địa chỉ I/O

+

10.5 Các loại hoạt động Intel x86 và ARM



Instruction	Description
Data Movement	
MOV	Move operand, between registers or between register and memory.
PUSH	Push operand onto stack.
PUSHA	Push all registers on stack.
MOVSX	Move byte, word, dword, sign extended. Moves a byte to a word or a word to a doubleword with twos-complement sign extension.
LEA	Load effective address. Loads the offset of the source operand, rather than its value to the destination operand.
XLAT	Table lookup translation. Replaces a byte in AL with a byte from a user-coded translation table. When XLAT is executed, AL should have an unsigned index to the table. XLAT changes the contents of AL from the table index to the table entry.
IN, OUT	Input, output operand from I/O space.
Arithmetic	
ADD	Add operands.
SUB	Subtract operands.
MUL	Unsigned integer multiplication, with byte, word, or double word operands, and word, doubleword, or quadword result.
IDIV	Signed divide.
Logical	
AND	AND operands.
BTS	Bit test and set. Operates on a bit field operand. The instruction copies the current value of a bit to flag CF and sets the original bit to 1.
BSF	Bit scan forward. Scans a word or doubleword for a 1-bit and stores the number of the first 1-bit into a register.
SHL/SHR	Shift logical left or right.
SAL/SAR	Shift arithmetic left or right.
ROL/ROR	Rotate left or right.
SETcc	Sets a byte to zero or one depending on any of the 16 conditions defined by status flags.
Control Transfer	
JMP	Unconditional jump.
CALL	Transfer control to another location. Before transfer, the address of the instruction following the CALL is placed on the stack.
JE/JZ	Jump if equal/zero.
LOOPE/LOOPZ	Loops if equal/zero. This is a conditional jump using a value stored in register ECX. The instruction first decrements ECX before testing ECX for the branch condition.
INT/INTO	Interrupt/Interrupt if overflow. Transfer control to an interrupt service routine

Bảng 12.8

Các loại hoạt động x86
(Với các ví dụ của các hoạt động thông thường)

(page 1 of 2)



String Operations	
MOVS	Move byte, word, dword string. The instruction operates on one element of a string, indexed by registers ESI and EDI. After each string operation, the registers are automatically incremented or decremented to point to the next element of the string.
LODS	Load byte, word, dword of string.
High-Level Language Support	
ENTER	Creates a stack frame that can be used to implement the rules of a block-structured high-level language.
LEAVE	Reverses the action of the previous ENTER.
BOUND	Check array bounds. Verifies that the value in operand 1 is within lower and upper limits. The limits are in two adjacent memory locations referenced by operand 2. An interrupt occurs if the value is out of bounds. This instruction is used to check an array index.
Flag Control	
STC	Set Carry flag.
LAHF	Load AH register from flags. Copies SF, ZF, AF, PF, and CF bits into A register.
Segment Register	
LDS	Load pointer into DS and another register.
	System Control
HLT	Halt.
LOCK	Asserts a hold on shared memory so that the Pentium has exclusive use of it during the instruction that immediately follows the LOCK.
ESC	Processor extension escape. An escape code that indicates the succeeding instructions are to be executed by a numeric coprocessor that supports high-precision integer and floating-point calculations.
WAIT	Wait until BUSY# negated. Suspends Pentium program execution until the processor detects that the BUSY pin is inactive, indicating that the numeric coprocessor has finished execution.
Protection	
SGDT	Store global descriptor table.
LSL	Load segment limit. Loads a user-specified register with a segment limit.
VERR/VERW	Verify segment for reading/writing.
Cache Management	
INVD	Flushes the internal cache memory.
WBINVD	Flushes the internal cache memory after writing dirty lines to memory.
INVLPG	Invalidates a translation lookaside buffer (TLB) entry.

Bảng 12.8

Các loại hoạt động x86

(Với các ví dụ của các hoạt động thông thường)

(page 2 of 2)

Các cờ trạng thái x86

Status Bit	Name	Description
CF	Carry	Indicates carrying or borrowing out of the left-most bit position following an arithmetic operation. Also modified by some of the shift and rotate operations.
PF	Parity	Parity of the least-significant byte of the result of an arithmetic or logic operation. 1 indicates even parity; 0 indicates odd parity.
AF	Auxiliary Carry	Represents carrying or borrowing between half-bytes of an 8-bit arithmetic or logic operation. Used in binary-coded decimal arithmetic.
ZF	Zero	Indicates that the result of an arithmetic or logic operation is 0.
SF	Sign	Indicates the sign of the result of an arithmetic or logic operation.
OF	Overflow	Indicates an arithmetic overflow after an addition or subtraction for twos complement arithmetic.



Symbol	Condition Tested	Comment
A, NBE	CF=0 AND ZF=0	Above; Not below or equal (greater than, unsigned)
AE, NB, NC	CF=0	Above or equal; Not below (greater than or equal, unsigned); Not carry
B, NAE, C	CF=1	Below; Not above or equal (less than, unsigned); Carry set
BE, NA	CF=1 OR ZF=1	Below or equal; Not above (less than or equal, unsigned)
E, Z	ZF=1	Equal; Zero (signed or unsigned)
G, NLE	[(SF=1 AND OF=1) OR (SF=0 and OF=0)] AND [ZF=0]	Greater than; Not less than or equal (signed)
GE, NL	(SF=1 AND OF=1) OR (SF=0 AND OF=0)	Greater than or equal; Not less than (signed)
L, NGE	(SF=1 AND OF=0) OR (SF=0 AND OF=1)	Less than; Not greater than or equal (signed)
LE, NG	(SF=1 AND OF=0) OR (SF=0 AND OF=1) OR (ZF=1)	Less than or equal; Not greater than (signed)
NE, NZ	ZF=0	Not equal; Not zero (signed or unsigned)
NO	OF=0	No overflow
NS	SF=0	Not sign (not negative)
NP, PO	PF=0	Not parity; Parity odd
O	OF=1	Overflow
P	PF=1	Parity; Parity even
S	SF=1	Sign (negative)

Bảng 12.10

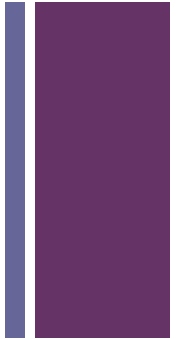
Các mã điều kiện x86 cho lệnh SETcc và lệnh nhảy có điều kiện



Ví dụ

1. Với mỗi số được mã hóa thập phân đóng như dưới đây, xác định giá trị số thập phân tương ứng:
 - a. 0111 0011 0000 1001
 - b. 0101 1000 0010
 - c. 0100 1010 0110
2. VXL có kích thước từ nhớ là 1 byte. Giá trị số nguyên lớn nhất và nhỏ nhất biểu diễn theo các dạng dưới đây là bao nhiêu
 - a. Số nguyên không dấu
 - b. Số nguyên dạng dấu – độ lớn
 - c. Số bù hai
 - d. Số thập phân đóng không dấu
 - e. Số thập phân đóng có dấu
3. Thực hiện dịch trái logic, dịch phải logic, dịch trái số học, dịch phải số học , vòng trái, vòng phải 4b với word 16b sau: 1001 1101 1100 0001

Các lệnh Đơn-lệnh, nhiều-dữ liệu x86 (Single Instruction – Multiple Data - SIMD Instructions)



- Năm 1996 Intel giới thiệu công nghệ MMX cho dòng VXL Pentium
 - MMX là một tập các lệnh được tối ưu hóa cao cho các chức năng đa phương tiện
- Các dữ liệu video và audio thường bao gồm các mảng lớn các kiểu dữ liệu nhỏ
- Ba kiểu dữ liệu mới được định nghĩa trong MMX
 - Packed Byte
 - Packed word
 - Packed doubleword
- Mỗi kiểu dữ liệu có kích thước 64 bit và gồm có nhiều trường dữ liệu nhỏ hơn, các trường này chứa dữ liệu số nguyên dấu phẩy thập



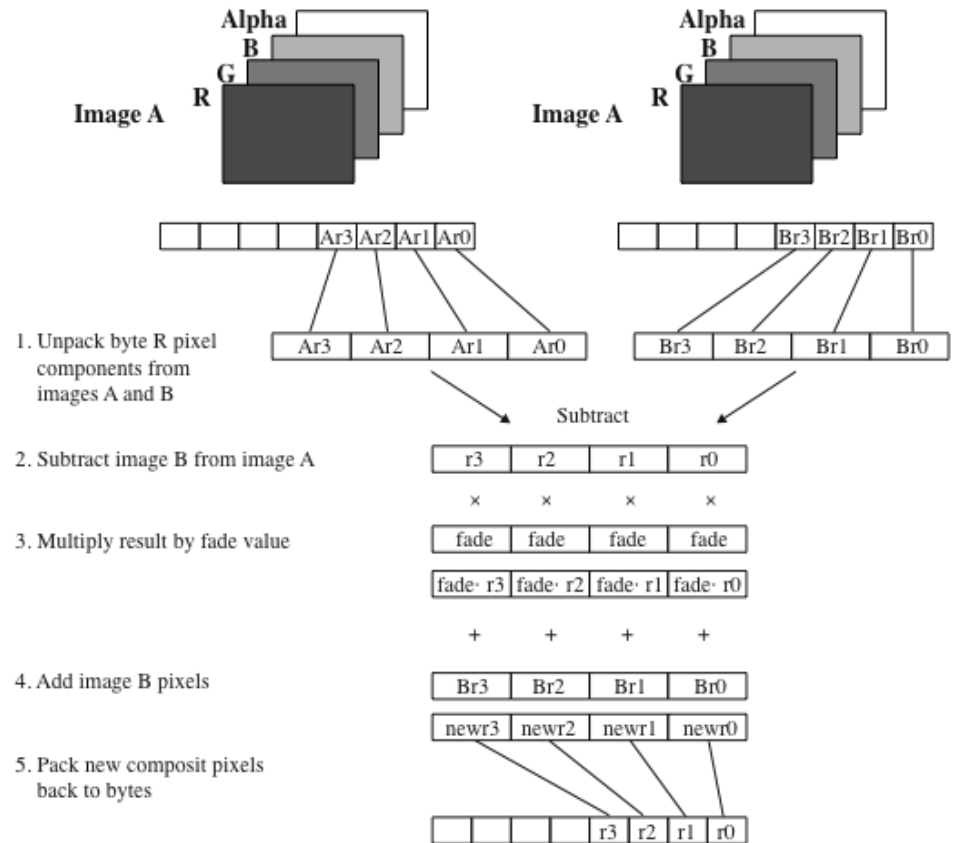
Tập lệnh MMX

Category	Instruction	Description
Arithmetic	PADD [B, W, D]	Parallel add of packed eight bytes, four 16-bit words, or two 32-bit doublewords, with wraparound.
	PADD [B, W]	Add with saturation.
	PADDUS [B, W]	Add unsigned with saturation
	PSUB [B, W, D]	Subtract with wraparound.
	PSUBS [B, W]	Subtract with saturation.
	PSUBUS [B, W]	Subtract unsigned with saturation
	PMULHW	Parallel multiply of four signed 16-bit words, with high-order 16 bits of 32-bit result chosen.
	PMULLW	Parallel multiply of four signed 16-bit words, with low-order 16 bits of 32-bit result chosen.
	PMADDWD	Parallel multiply of four signed 16-bit words; add together adjacent pairs of 32-bit results.
Comparison	PCMPEQ [B, W, D]	Parallel compare for equality; result is mask of 1s if true or 0s if false.
	PCMPGT [B, W, D]	Parallel compare for greater than; result is mask of 1s if true or 0s if false.
Conversion	PACKUSWB	Pack words into bytes with unsigned saturation.
	PACKSS [WB, DW]	Pack words into bytes, or doublewords into words, with signed saturation.
	PUNPCKH [BW, WD, DQ]	Parallel unpack (interleaved merge) high-order bytes, words, or doublewords from MMX register.
	PUNPCKL [BW, WD, DQ]	Parallel unpack (interleaved merge) low-order bytes, words, or doublewords from MMX register.
Logical	PAND	64-bit bitwise logical AND
	PANDN	64-bit bitwise logical AND NOT
	POR	64-bit bitwise logical OR
	PXOR	64-bit bitwise logical XOR
Shift	PSLL [W, D, Q]	Parallel logical left shift of packed words, doublewords, or quadword by amount specified in MMX register or immediate value.
	PSRL [W, D, Q]	Parallel logical right shift of packed words, doublewords, or quadword.
	PSRA [W, D]	Parallel arithmetic right shift of packed words, doublewords, or quadword.
Data Transfer	MOV [D, Q]	Move doubleword or quadword to/from MMX register.
State Mgt	EMMS	Empty MMX state (empty FP registers tag bits).

Note: If an instruction supports multiple data types [byte (B), word (W), doubleword (D), quadword (Q)], the data types are indicated in brackets.



Image Compositing on Color Plane Representation



MMX code sequence performing this operation:

```
pxor      mm7, mm7      ;zero out mm7
movq      mm3, fad_val  ;load fade value replicated 4 times
```

Các loại câu lệnh ARM

Các lệnh tải
và lưu trữ dữ
liệu

Các lệnh rẽ
nhánh

Các lệnh xử
lý dữ liệu

Các lệnh
nhân

Các lệnh
cộng trừ
song song

Các lệnh mở
rộng

Các lệnh truy
cập vào thanh
ghi trạng thái



Code	Symbol	Condition Tested	Comment
0000	EQ	$Z = 1$	Equal
0001	NE	$Z = 0$	Not equal
0010	CS/HS	$C = 1$	Carry set/unsigned higher or same
0011	CC/LO	$C = 0$	Carry clear/unsigned lower
0100	MI	$N = 1$	Minus/negative
0101	PL	$N = 0$	Plus/positive or zero
0110	VS	$V = 1$	Overflow
0111	VC	$V = 0$	No overflow
1000	HI	$C = 1 \text{ AND } Z = 0$	Unsigned higher
1001	LS	$C = 0 \text{ OR } Z = 1$	Unsigned lower or same
1010	GE	$N = V$ $[(N = 1 \text{ AND } V = 1)$ $\text{OR } (N = 0 \text{ AND } V = 0)]$	Signed greater than or equal
1011	LT	$N \neq V$ $[(N = 1 \text{ AND } V = 0)$ $\text{OR } (N = 0 \text{ AND } V = 1)]$	Signed less than
1100	GT	$(Z = 0) \text{ AND } (N = V)$	Signed greater than
1101	LE	$(Z = 1) \text{ OR } (N \neq V)$	Signed less than or equal
1110	AL	—	Always (unconditional)
1111	—	—	This instruction can only be executed unconditionally

Các điều kiện ARM cho thực thi câu lệnh điều kiện

+ Tổng kết

Chương 10

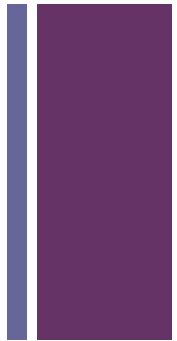
- Đặc điểm lệnh máy
 - Các thành phần của lệnh máy
 - Biểu diễn lệnh
 - Các loại lệnh
 - Number of addresses
 - Thiết kế tập lệnh
- Các loại toán hạng
 - Số
 - Ký tự
 - Dữ liệu logic

Tập lệnh: Đặc điểm và chức năng

- Các kiểu dữ liệu Intel x86 và ARM
- Các loại hoạt động
 - Truyền dữ liệu
 - Tính toán số học
 - Logical
 - Chuyển đổi
 - Vào/ra
 - Điều khiển hệ thống
 - Truyền điều khiển
- Các loại hoạt động trong Intel x86 và ARM



Câu hỏi chương 10



- 1 Các thành phần điển hình của một lệnh máy?
- 2 Toán hạng nguồn và đích có thể được đặt ở đâu?
- 3 Nếu một lệnh có 4 địa chỉ, mục đích của từng địa chỉ là gì?
- 4 Trình bày ngắn gọn 5 vấn đề thiết kế tập lệnh quan trọng.
- 5 Các loại toán hạng điển hình trong tập lệnh máy?
- 6 Mối quan hệ giữa mã ký tự IRA và biểu diễn packed decimal?
- 7 Phân biệt dịch số học và dịch logic?
- 8 Tại sao phải truyền lệnh điều khiển?
- 9 Trình bày hai cách tạo điều kiện để kiểm tra một lệnh rẽ nhánh
- 10 Ý nghĩa của thuật ngữ làm thủ tục lồng nhau là gì?
- 11 Liệt kê ba vị trí có thể lưu trữ địa chỉ trả về của thủ tục có trả về.
- 12 Phân biệt endian lớn và endian nhỏ?